NyotaPay Payment Platform

Enterprise Architecture Document v2.0

Document Status: Final
Date: November 5, 2025
Author: Architecture Team
Classification: Confidential

Executive Summary

NyotaPay is a comprehensive payment platform designed for on-premises deployment, supporting P2P transfers, cash-in/out operations, and merchant payments across multiple channels (Mobile, USSD, SMS). This document presents an improved architecture leveraging modern design patterns, cloud-native principles, and financial services best practices.

Key Improvements

- **Microservices with Domain-Driven Design (DDD)**: Clear bounded contexts and service boundaries
- **Event-Driven Architecture**: CQRS and Event Sourcing for scalability and auditability
- **Enhanced Security**: Zero-trust architecture, service mesh, and advanced threat protection
- **Cloud-Native Patterns**: Container orchestration, service mesh, and infrastructure as code
- **Improved Resilience**: Circuit breakers, bulkheads, retry policies, and chaos engineering
- **Advanced Observability**: Distributed tracing, metrics, and centralized logging

Table of Contents

1. Architecture Principles

1.1 Core Principles

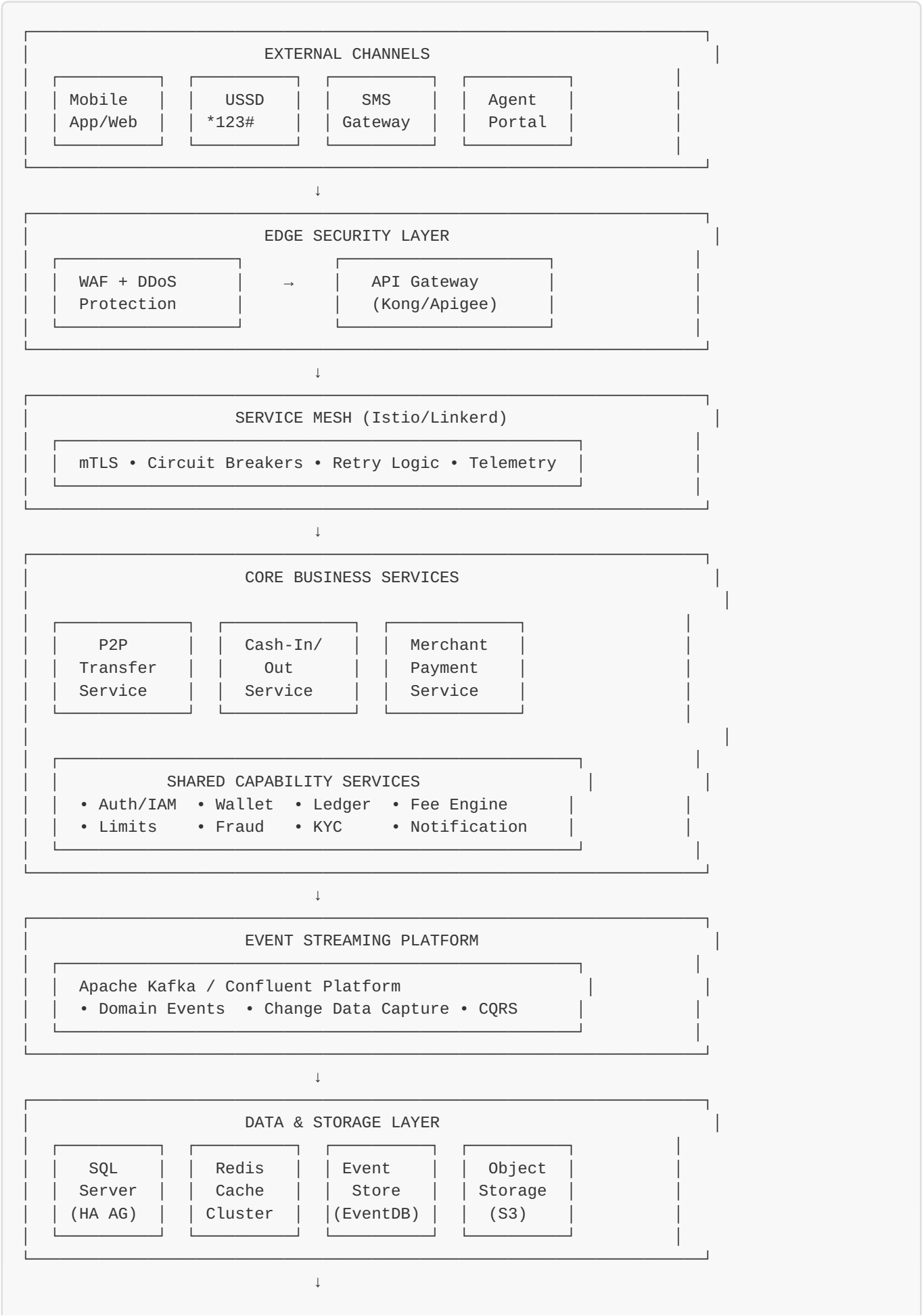
1. **Security First**: Every component implements defense-in-depth
2. **Financial Integrity**: Strong consistency for money movements, eventual consistency elsewhere
3. **Scalability by Design**: Horizontal scaling, stateless services
4. **Resilience**: Graceful degradation, circuit breakers, bulkheads
5. **Observability**: Full traceability of every transaction
6. **Domain-Driven Design**: Business logic encapsulated in bounded contexts
7. **API-First**: All functionality exposed through well-defined APIs
8. **Eventual Consistency**: Event-driven architecture for loose coupling

1.2 Technology Principles

- **Polyglot Persistence**: Right database for the right use case
- **Infrastructure as Code**: All infrastructure versioned and automated
- **Immutable Infrastructure**: Container-based deployments
- **GitOps**: Declarative infrastructure and application deployment
- **Automated Testing**: Unit, integration, contract, and chaos testing

2. System Context

2.1 High-Level Architecture


EXTERNAL CHANNELS
Mobile App/Web
USSD *123#
SMS Gateway
Agent Portal
EDGE SECURITY LAYER
WAF + DDoS Protection
API Gateway (Kong/Apigee)
SERVICE MESH (Istio/Linkerd)
mTLS • Circuit Breakers • Retry Logic • Telemetry
CORE BUSINESS SERVICES
P2P Transfer Service
Cash-In/ Out Service
Merchant Payment Service
SHARED CAPABILITY SERVICES
• Auth/IAM • Wallet • Ledger • Fee Engine
• Limits • Fraud • KYC • Notification
EVENT STREAMING PLATFORM
Apache Kafka / Confluent Platform
• Domain Events • Change Data Capture • CQRS
DATA & STORAGE LAYER
SQL Server (HA AG)
Redis Cache Cluster
Event Store (EventDB)
Object Storage (S3)

2.2 Service Boundaries (Bounded Contexts)

1. **Payment Processing Context**
   - P2P Transfer Service
   - Cash-In/Out Service
   - Merchant Payment Service
2. **Account Management Context**
   - Wallet Service
   - Customer Service
   - KYC Service
3. **Financial Context**
   - Ledger Service
   - Fee Calculation Service
   - Settlement Service
4. **Risk Management Context**
   - Fraud Detection Service
   - Limits Enforcement Service
   - AML/Compliance Service
5. **Notification Context**
   - Notification Service (SMS, Push, Email)
   - Communication Gateway
6. **Integration Context**
   - External Partner Adapters
   - Channel Gateway Service

3. Architecture Patterns

3.1 CQRS (Command Query Responsibility Segregation)

Command Side (Write Model)
- Handles all state changes

- Strong consistency
- Domain-driven design
- Event sourcing optional

Query Side (Read Model)
- Optimized for reads
- Eventual consistency
- Denormalized views
- Multiple read models per service

```
Command → Aggregate → Event → Event Store → Projection → Read Model
```

3.2 Event Sourcing

Store all state changes as sequence of events in an Event Store. Each event represents a state change with:
- Aggregate identifier
- Version number for optimistic concurrency
- Event type and payload
- Metadata and timestamp

Benefits:
- Complete audit trail
- Time travel debugging
- Event replay for new projections
- Temporal queries

3.3 Saga Pattern

For distributed transactions across services:

Orchestration-Based Saga:

```
P2P Transfer Saga:
1. Reserve sender balance
2. Create transfer record
3. Credit receiver wallet
4. Post ledger entries
5. Send notifications
```

Compensation Logic:
Each step has compensating transaction for rollback.

3.4 Outbox Pattern

Ensures reliable event publishing by writing events to an outbox table in the same transaction as business data. The process:

1. Business transaction writes data to main tables
2. Same transaction writes events to outbox table
3. Transaction commits atomically
4. Background worker polls outbox and publishes events to message broker
5. Mark events as processed after successful publication

3.5 API Gateway Pattern

Single entry point with:
- Request routing
- Protocol translation
- Composition
- Authentication/Authorization
- Rate limiting
- Caching

3.6 Service Discovery Pattern

Service registry for dynamic service location:
- Automatic service registration on startup
- Health-based service discovery
- DNS and API-based lookups
- Configuration distribution

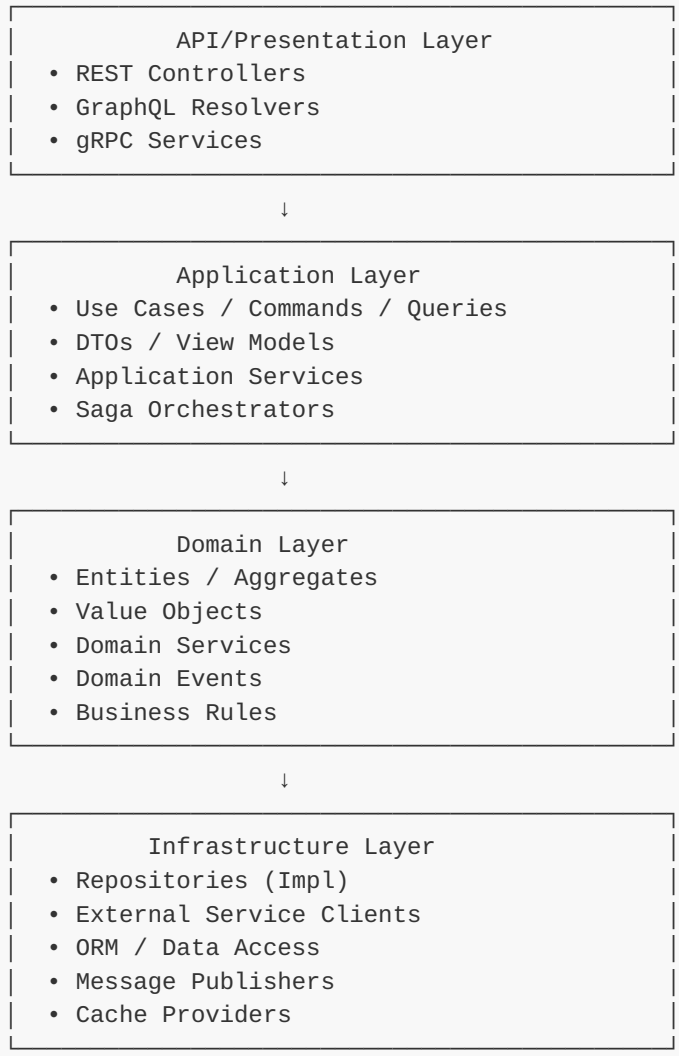
3.7 Strangler Fig Pattern

For gradual migration:

```
Legacy System → Proxy → Route to New/Old Service
```

4. Component Architecture

4.1 Service Structure (Clean Architecture)



4.2 P2P Transfer Service (Detailed)

Command Flow:

```
POST /api/v1/transfers/p2p

1. TransferController.InitiateP2P()
   ↓
2. TransferCommandHandler.Handle()
   - Validate request
   - Check idempotency
   ↓
3. TransferSaga.Execute()
   - Reserve sender balance
   - Verify receiver account
   - Apply limits
   - Check fraud rules
   - Calculate fees
   ↓
4. TransferAggregate.Execute()
   - Domain validation
   - Emit TransferInitiated event
   - Emit BalanceReserved event
   ↓
5. Repository.Save()
   - Write to event store
   - Write to outbox
   - Update read model
   ↓
6. Return 202 Accepted
   TransferId + Status URL
```

Event Handlers:

- `TransferCompleted` → Update wallet balances
- `TransferCompleted` → Post ledger entries
- `TransferCompleted` → Send notifications
- `TransferCompleted` → Update analytics

4.3 Ledger Service

Double-Entry Bookkeeping:

The ledger service maintains a complete financial record using double-entry accounting principles. Each ledger entry contains:

- Unique entry identifier
- Transaction reference
- Account identifier
- Debit or credit amount (never both)
- Running balance after entry
- Entry type and reference information
- Timestamp

For each transfer, the system creates paired entries:

- Entry 1: Debit sender account

- Entry 2: Credit receiver account
- Entry 3: Debit receiver for fee (optional)
- Entry 4: Credit fee income account (optional)

Database constraints ensure each entry has either a debit OR credit amount, never both or neither.

Consistency:
- All entries in single transaction
- Running balance calculation
- Immutable ledger (no updates/deletes)

4.4 Wallet Service

Capabilities:
- Balance queries (cache-first)
- Account status management
- Balance reservations
- Multi-currency support

Balance Calculation:

The wallet service implements a cache-first strategy for balance queries:
1. Check cache for account balance using account ID as key
2. If cached value exists, return immediately
3. If cache miss, query ledger service to compute balance from ledger entries
4. Cache the computed balance with 30-second TTL
5. Return balance to caller

This approach reduces database load while ensuring balance data remains reasonably fresh.

4.5 Authentication & Authorization Service

OAuth 2.0 / OpenID Connect:
- Client credentials for service-to-service
- Authorization code for user apps
- JWT access tokens
- Refresh tokens

Token Structure:

```
{
  "sub": "customer:550e8400",
  "iss": "https://auth.nyotapay.com",
  "aud": ["api.nyotapay.com"],
  "exp": 1730000000,
  "iat": 1729996400,
  "scope": ["transfer:create", "wallet:read"],
  "roles": ["customer"],
  "kyc_level": "tier2"
}
```

RBAC + ABAC:
- Role-Based Access Control for coarse permissions
- Attribute-Based for fine-grained (e.g., KYC level)

4.6 Fraud Detection Service

Real-Time Fraud Checks:
1. Velocity checks (transaction frequency)
2. Amount anomaly detection
3. Geo-location analysis
4. Device fingerprinting
5. Behavioral biometrics
6. ML-based risk scoring

Decision Engine:

```
Risk Score < 30: Auto-approve
Risk Score 30-70: Manual review
Risk Score > 70: Auto-reject + Alert
```

4.7 Fee Calculation Engine

Dynamic Fee Structure:

The fee calculation engine uses a rule-based approach:

Input Parameters:
- Transaction type (P2P, cash-in, merchant payment)
- Transaction amount
- Customer tier (basic, silver, gold, platinum)
- Merchant category (if applicable)

Calculation Process:
1. Rule engine matches input parameters to appropriate fee rule
2. Apply fee calculation based on rule type:
- **Percentage**: Fee = Amount × Rate

- **Fixed**: Fee = Fixed Amount
- **Tiered**: Fee calculated based on amount brackets
3. Apply maximum fee cap if defined
4. Calculate tax amount based on fee and tax rate
5. Return complete fee breakdown

Fee Result:
- Fee amount (before tax)
- Tax amount
- Net fee (total charged to customer)

---

5. Security Architecture

5.1 Defense in Depth

Layer 1: Network Security
- VPC / Network segmentation
- Firewall rules
- DDoS protection
- IDS/IPS

Layer 2: Edge Security
- WAF (ModSecurity/Cloudflare)
- Rate limiting
- IP allowlisting
- Bot protection

Layer 3: API Gateway
- Authentication (OAuth 2.0)
- Authorization (RBAC/ABAC)
- API key validation
- Request validation

Layer 4: Service Mesh
- mTLS between all services
- Service identity (SPIFFE)
- Fine-grained access control

Layer 5: Application
- Input validation
- SQL injection prevention
- XSS protection
- CSRF tokens

Layer 6: Data
- Encryption at rest (AES-256)
- Encryption in transit (TLS 1.3)
- Field-level encryption (PII)
- Key rotation

5.2 Zero Trust Architecture

Principles:
1. Never trust, always verify
2. Least privilege access
3. Microsegmentation
4. Continuous monitoring

Implementation:

```
Request → Identity Verification → Device Posture Check
  → Context Evaluation → Policy Decision → Allow/Deny
```

5.3 Secrets Management

HashiCorp Vault / Azure Key Vault:

```
Secrets Storage:
- Database credentials
- API keys
- Certificates
- Encryption keys

Features:
- Dynamic secrets (short-lived)
- Automatic rotation
- Audit logging
- Access policies
```

5.4 PCI DSS Compliance

Requirements Mapping:
- **Req 1-2:** Network segmentation, no default passwords
- **Req 3-4:** Encryption, secure transmission
- **Req 5-6:** Anti-malware, secure development
- **Req 7-9:** Access control, physical security
- **Req 10-12:** Logging, security policies

5.5 Cryptographic Standards

Encryption:
- Symmetric: AES-256-GCM
- Asymmetric: RSA-4096, ECC P-384
- Hashing: SHA-256, bcrypt (passwords)
- MAC: HMAC-SHA256

Key Management:
- Hardware Security Modules (HSM)
- Key rotation every 90 days
- Separate keys per environment

---

6. Data Architecture

6.1 Polyglot Persistence

SQL Server (Primary Transactional Data):
- Customer accounts
- Transactions
- Ledger entries
- KYC documents

Redis (Caching & Session):
- Session state
- Balance cache (30s TTL)
- Rate limit counters
- Distributed locks

Event Store (Event Sourcing):
- EventStoreDB or Kafka
- Immutable event log
- Aggregate event streams

Time-Series DB (Metrics):
- Prometheus
- Transaction metrics
- Performance data

Document Store (Optional - MongoDB):
- Audit logs
- Analytics data
- Configuration

Object Storage (S3/MinIO):
- KYC documents
- Transaction receipts
- Backups

6.2 Database Schema Design

Transaction Table:

The transaction table stores all payment transactions with the following key attributes:
- Unique transaction identifier (GUID)
- External reference (unique, for idempotency)
- Transaction type (P2P, cash-in, cash-out, merchant payment)
- Source and destination account identifiers
- Amount and currency
- Fee amount
- Status (pending, processing, completed, failed)
- Failure reason (if applicable)
- Metadata (JSON format for flexible attributes)
- Timestamps (created, updated, completed)
- Idempotency key for duplicate detection

Indexes are created on:
- External reference for fast lookup
- Creation timestamp for time-based queries
- Status for filtering active transactions

Account Table:

The account table manages customer wallet accounts with:
- Unique account identifier (GUID)
- Customer identifier (foreign key)
- Account number (unique)
- Account type (personal, business, agent)
- Currency
- Status (active, frozen, closed)
- Computed balance (calculated from ledger entries)
- Creation timestamp

The balance is computed dynamically from ledger entries to ensure consistency with the ledger of record.

Outbox Table:

The outbox table facilitates reliable event publishing with:
- Auto-incrementing outbox identifier
- Aggregate identifier
- Event type
- Event payload (JSON)

- Processing status (pending, processed, failed)
- Retry count
- Timestamps (created, processed)

An index on status and creation time enables efficient polling for pending events.

6.3 Data Consistency Patterns

Strong Consistency (Synchronous):
- Wallet balance updates
- Ledger postings
- Transaction status

Eventual Consistency (Asynchronous):
- Notifications
- Analytics
- Reporting
- Search indexes

Saga Pattern for Distributed Transactions:

```
Transfer Saga:
1. Reserve sender balance (compensate: Unreserve)
2. Validate receiver (compensate: N/A)
3. Create transaction (compensate: Cancel transaction)
4. Post to ledger (compensate: Reverse entry)
5. Send notification (no compensation)
```

6.4 Data Retention & Archival

Hot Storage (SSD): Last 90 days
Warm Storage (HDD): 91 days - 7 years
Cold Storage (S3 Glacier): > 7 years

Regulatory Requirements:
- Transaction data: 7 years (PCI DSS)
- KYC documents: 5 years after account closure
- Audit logs: 1 year hot, 7 years cold

7. Integration Architecture

7.1 Integration Patterns

Adapter Pattern:

```
Core Service → Interface → Adapter → External System
```

Each adapter implements a standard interface with the following operations:
- **ProcessPayment**: Initiates a payment transaction with the external gateway
- **GetStatus**: Queries the current status of a transaction using reference ID
- **RefundPayment**: Processes a refund for a completed transaction

All adapters follow the same contract for consistent integration patterns across different payment providers.

7.2 Bank Integration

ISO 8583 Messaging:

```
MTI: 0200 (Financial Transaction Request)
Fields:
  2: PAN
  3: Processing Code
  4: Amount
  7: Transmission Date/Time
  11: STAN
  37: Reference Number
  ...
```

Async Settlement:

```
1. Real-time authorization
2. Batch settlement (EOD)
3. Reconciliation file exchange
4. Settlement confirmation
```

7.3 Telco Integration

USSD Gateway:

```
User → *123# → Telco USSD Gateway → NyotaPay USSD Service
       ← Menu Response ←
```

Mobile Money Integration:
- REST API (MTN, Airtel, Vodacom)
- OAuth 2.0 authentication
- Webhook callbacks for status

Airtime Topup:

```
Request → Adapter → Telco API → Instant Topup
Response ← Status ← Callback ←
```

7.4 Merchant Integration

Payment Gateway SDK:

```
const nyotaPay = new NyotaPay({ apiKey: 'xxx' });

const payment = await nyotaPay.createPayment({
  amount: 50000,
  currency: 'TZS',
  merchantRef: 'ORDER-12345',
  callbackUrl: 'https://merchant.com/callback'
});

// Redirect customer to payment.checkoutUrl
```

Webhook for Status:

```
POST /merchant/callback
{
  "eventType": "payment.completed",
  "paymentId": "550e8400-e29b-41d4-a716-446655440000",
  "merchantRef": "ORDER-12345",
  "amount": 50000,
  "status": "COMPLETED",
  "timestamp": "2025-11-05T10:30:00Z",
  "signature": "sha256=abc123..."
}
```

8. Deployment Architecture

8.1 VM-Based Deployment Architecture

Infrastructure Layout:

```
Production Environment:
├── Load Balancer Tier (3 VMs)
│   ├── HAProxy / Nginx Load Balancers
│   ├── Keepalived for HA (Virtual IP failover)
│   └── API Gateway instances
│
├── Application Tier (12 VMs)
│   ├── P2P Service VMs (3x)
│   ├── Cash-In/Out Service VMs (3x)
│   ├── Merchant Payment Service VMs (3x)
│   └── Shared Services VMs (3x)
│       ├── Wallet Service
│       ├── Ledger Service
│       └── Auth Service
│
├── Integration Tier (4 VMs)
│   ├── Bank Adapters (2x)
│   └── Telco/Merchant Adapters (2x)
│
├── Data Tier
│   ├── SQL Server Cluster (3 VMs - AlwaysOn AG)
│   ├── Redis Cluster (6 VMs - 3 masters, 3 replicas)
│   └── Kafka Cluster (5 VMs + 3 Zookeeper VMs)
│
└── Platform Services Tier (6 VMs)
    ├── Monitoring (Prometheus, Grafana - 2 VMs)
    ├── Logging (ELK Stack - 3 VMs)
    └── Secrets Management (Vault - 1 VM)
```

VM Organization:
- **Core Services**: Business logic services in isolated VMs
- **Shared Services**: Common capability services
- **Integration**: External system adapters
- **Platform**: Infrastructure and observability
- **Data**: Database and messaging infrastructure

8.2 Service Deployment on VMs

VM Specifications:

P2P Service VM:
- **OS**: Ubuntu 22.04 LTS / RHEL 8
- **CPU**: 4 vCPU
- **Memory**: 8 GB RAM
- **Disk**: 100 GB SSD
- **Network**: 1 Gbps, multiple NICs (management, application, data)

Service Deployment Process:
1. Provision VM using automated templates
2. Configure OS hardening and security policies
3. Install runtime dependencies (.NET 8 Runtime)

4. Deploy application binaries from artifact repository
5. Configure service as systemd unit for auto-restart
6. Set environment variables from configuration management
7. Configure health check endpoints
8. Register service with service discovery (Consul/etcd)
9. Add VM to load balancer pool

Systemd Service Configuration:

```
[Unit]
Description=NyotaPay P2P Transfer Service
After=network.target

[Service]
Type=notify
User=nyotapay
Group=nyotapay
WorkingDirectory=/opt/nyotapay/p2p-service
ExecStart=/usr/bin/dotnet /opt/nyotapay/p2p-service/NyotaPay.P2P.dll
Restart=always
RestartSec=10
Environment="ASPNETCORE_ENVIRONMENT=Production"
Environment="DATABASE_URL=<from-vault>"

# Resource limits
MemoryLimit=8G
CPUQuota=400%

# Health checks
TimeoutStartSec=60
WatchdogSec=30

[Install]
WantedBy=multi-user.target
```

Health Check Configuration:
- **Liveness**: HTTP GET /health/live every 10 seconds
- **Readiness**: HTTP GET /health/ready every 5 seconds
- **Startup**: Initial delay 30 seconds

8.3 Service Discovery and Load Balancing

Service Discovery (Consul):

Benefits:
- Automatic service registration
- Health-based routing
- DNS-based service discovery
- Key-value store for configuration

Service Registration:

Each service automatically registers with Consul on startup:

- Service name and version
- IP address and port
- Health check endpoints
- Metadata (environment, datacenter, tags)

Consul Agent Configuration:

```
{
  "service": {
    "name": "p2p-service",
    "tags": ["production", "v1.2.0"],
    "port": 8080,
    "check": {
      "http": "http://localhost:8080/health/ready",
      "interval": "10s",
      "timeout": "2s"
    }
  }
}
```

Load Balancing with HAProxy:

Features:

- Layer 7 load balancing
- SSL termination
- Health checks
- Session persistence
- Traffic shaping

HAProxy Configuration:

```
frontend api_frontend
    bind *:443 ssl crt /etc/ssl/certs/nyotapay.pem
    mode http
    option httplog
    default_backend p2p_backend

backend p2p_backend
    mode http
    balance roundrobin
    option httpchk GET /health/ready
    http-check expect status 200

    server p2p-vm-1 10.0.1.11:8080 check inter 5s fall 3 rise 2
    server p2p-vm-2 10.0.1.12:8080 check inter 5s fall 3 rise 2
    server p2p-vm-3 10.0.1.13:8080 check inter 5s fall 3 rise 2

    # Retry configuration
    retries 3
    timeout connect 5s
    timeout server 30s
```

Keepalived for HA:

Multiple HAProxy instances with Virtual IP failover:

- Master HAProxy holds Virtual IP
- Backup HAProxy monitors master via VRRP
- Automatic failover in <1 second

8.4 Infrastructure as Code

Terraform for VM Provisioning:

```
# P2P Service VM Pool
resource "vsphere_virtual_machine" "p2p_service" {
  count            = 3
  name             = "p2p-vm-${count.index + 1}"
  resource_pool_id = data.vsphere_resource_pool.pool.id
  datastore_id     = data.vsphere_datastore.datastore.id

  num_cpus = 4
  memory   = 8192

  network_interface {
    network_id = data.vsphere_network.network.id
  }

  disk {
    label            = "disk0"
    size             = 100
    thin_provisioned = true
  }

  clone {
    template_uuid = data.vsphere_virtual_machine.template.id

    customize {
      linux_options {
        host_name = "p2p-vm-${count.index + 1}"
        domain    = "nyotapay.local"
      }

      network_interface {
        ipv4_address = "10.0.1.${11 + count.index}"
        ipv4_netmask = 24
      }

      ipv4_gateway = "10.0.1.1"
    }
  }

  # Provision with Ansible
  provisioner "local-exec" {
    command = "ansible-playbook -i ${self.default_ip_address}, playbooks/p2p-service.yml"
  }
}

# Load Balancer VMs
resource "vsphere_virtual_machine" "load_balancer" {
  count  = 3
  name   = "lb-vm-${count.index + 1}"
  # ... similar configuration
}

# Application Load Balancer configuration
resource "null_resource" "configure_haproxy" {
  depends_on = [vsphere_virtual_machine.p2p_service]

  provisioner "local-exec" {
```

```
    command = "ansible-playbook playbooks/haproxy-config.yml"
  }
}
```

Ansible for Configuration Management:

```
# playbooks/p2p-service.yml
---
- name: Deploy P2P Service
  hosts: all
  become: yes

  roles:
    - common
    - dotnet-runtime
    - consul-agent
    - p2p-service
    - monitoring-agent

  tasks:
    - name: Create application user
      user:
        name: nyotapay
        system: yes

    - name: Deploy application binaries
      copy:
        src: "{{ artifact_path }}/p2p-service/"
        dest: /opt/nyotapay/p2p-service/
        owner: nyotapay
        group: nyotapay

    - name: Configure systemd service
      template:
        src: p2p-service.service.j2
        dest: /etc/systemd/system/p2p-service.service

    - name: Start and enable service
      systemd:
        name: p2p-service
        state: started
        enabled: yes
        daemon_reload: yes
```

Configuration Management:

- Ansible for VM configuration and application deployment
- Consul for service discovery and configuration distribution
- Vault for secrets management
- Git for version control of all configurations

8.5 High Availability Setup

SQL Server:

```
AlwaysOn Availability Group:
├── Primary Replica (Node 1) - Read/Write
├── Secondary Replica (Node 2) - Async Sync + Readable
└── Secondary Replica (Node 3) - Async Sync + Readable

Quorum: Node + File Share Witness
Listener: sql-nyotapay-ag.local:1433
```

Redis:

```
Redis Cluster (6 nodes):
├── Master 1 (Slots: 0-5460)
├── Master 2 (Slots: 5461-10922)
├── Master 3 (Slots: 10923-16383)
└── 3x Replicas (one per master)

Sentinel for automatic failover
```

Kafka:

```
Kafka Cluster (5 brokers):
├── Broker 1-3: Topic partitions
├── Broker 4-5: Additional replicas
└── Zookeeper Ensemble (3 nodes)

Replication Factor: 3
Min In-Sync Replicas: 2
```

9. Resilience & Scalability

9.1 Resilience Patterns

Circuit Breaker:
Prevents cascading failures by stopping calls to failing services:
- **Closed State**: Requests pass through normally
- **Open State**: After 5 consecutive failures, circuit opens for 30 seconds
- **Half-Open State**: After timeout, allow test request to check service health
- Log circuit state changes for monitoring

Retry Policy:
Automatically retries transient failures with exponential backoff:
- Retry failed requests up to 3 times
- Wait duration: 2^attempt seconds (2s, 4s, 8s)
- Log each retry attempt with timing
- Handle transient errors like network issues

Bulkhead Isolation:
Limits concurrent operations to prevent resource exhaustion:
- Maximum 10 parallel operations
- Queue up to 20 additional requests
- Reject requests beyond queue capacity
- Track rejection metrics for capacity planning

Timeout:
Prevents indefinite waiting for slow operations:
- 10-second timeout for external service calls
- Pessimistic strategy (cancels operation actively)
- Fail fast rather than hanging

Combined Policy:
Multiple resilience patterns work together:
1. Timeout ensures operations don't hang
2. Bulkhead limits concurrent load
3. Retry handles transient failures
4. Circuit breaker prevents repeated failures to unhealthy services

All policies are applied in a layered approach for comprehensive resilience.

9.2 Scalability Strategies

Horizontal Scaling (Add/Remove VMs):

Automated VM Scaling with vSphere API:

Scaling Triggers:
- CPU utilization > 70% for 5 minutes → Scale up
- CPU utilization < 30% for 10 minutes → Scale down
- Memory utilization > 80% → Scale up
- Request queue depth > 1000 → Scale up

Scaling Process:
1. **Monitoring**: Prometheus collects metrics from all service VMs
2. **Evaluation**: Alert manager evaluates scaling rules
3. **Decision**: Scaling script triggered via webhook
4. **Provision**: Terraform/Ansible provisions new VM from template
5. **Configure**: Ansible configures application and dependencies
6. **Register**: Service registers with Consul automatically

7. **Load Balance**: HAProxy detects new service via Consul
8. **Traffic**: New VM starts receiving traffic

Scaling Configuration:

```
# scaling-rules.yml
p2p_service:
  min_vms: 3
  max_vms: 10
  scale_up:
    cpu_threshold: 70
    memory_threshold: 80
    evaluation_period: 5m
    cooldown: 10m
  scale_down:
    cpu_threshold: 30
    memory_threshold: 50
    evaluation_period: 10m
    cooldown: 15m
```

Scaling Script:
- Monitor Prometheus metrics
- Execute Terraform to provision/destroy VMs
- Update load balancer configuration
- Drain connections before removing VMs

Vertical Scaling (Resize VMs):

VM Resizing Process:
1. Identify under-resourced VMs through metrics
2. Schedule maintenance window
3. Gracefully drain traffic from VM
4. Power off VM
5. Increase CPU/Memory allocation
6. Power on VM
7. Re-register with service discovery
8. Resume traffic

Resize Automation:
- Automated during off-peak hours
- Manual approval for production
- Zero-downtime with multiple VM instances

Database Scaling:
- Read replicas for read-heavy queries
- Connection pooling (min: 10, max: 100)
- Query caching
- Database sharding (future)

Cache Scaling:
- Redis Cluster mode (horizontal scaling)
- Cache-aside pattern
- Write-through caching
- TTL-based invalidation

9.3 Load Balancing

Layer 7 Load Balancer (Nginx):

```
upstream p2p_service {
    least_conn;
    server p2p-1.core.svc.cluster.local:8080 weight=3;
    server p2p-2.core.svc.cluster.local:8080 weight=3;
    server p2p-3.core.svc.cluster.local:8080 weight=3;

    keepalive 32;
}

server {
    listen 443 ssl http2;
    server_name api.nyotapay.com;

    ssl_certificate /etc/ssl/certs/nyotapay.crt;
    ssl_certificate_key /etc/ssl/private/nyotapay.key;

    location /api/v1/transfers {
        proxy_pass http://p2p_service;
        proxy_http_version 1.1;
        proxy_set_header Connection "";
        proxy_set_header Host $host;
        proxy_set_header X-Request-ID $request_id;
    }
}
```

9.4 Rate Limiting

Distributed Rate Limiting (Redis):

Uses Redis for distributed rate limiting across multiple application instances:
1. Create unique key combining client ID and current minute timestamp
2. Increment counter for this key in Redis
3. Set 1-minute expiration on first increment
4. Check if count exceeds limit (e.g., 100 requests per minute)
5. Allow or reject request based on count

This approach ensures consistent rate limiting across all service instances.

Token Bucket Algorithm:

Implements smooth rate limiting with burst capacity:

Configuration:
- Bucket capacity (maximum tokens)
- Refill rate (tokens per second)

Operation:
1. **Refill**: Calculate tokens to add based on elapsed time since last refill
2. Add tokens to bucket (up to capacity limit)
3. **Consume**: Check if enough tokens available
4. If sufficient tokens, deduct and allow request
5. If insufficient, reject request

Benefits:
- Allows burst traffic up to capacity
- Smooth refill prevents thundering herd
- Fair distribution over time

9.5 Chaos Engineering

Chaos Engineering Experiments:

VM Failure Testing:
- Randomly shutdown service VMs during business hours
- Verify automatic failover and traffic rerouting
- Measure recovery time and customer impact
- Tools: Custom scripts, vSphere API

Network Chaos Testing:
- Inject network latency using Linux tc (traffic control)
- Simulate packet loss and network partitions
- Test service behavior under degraded network conditions
- Verify circuit breakers and timeouts work correctly

Example Network Latency Injection:

```
# Add 200ms latency with 50ms jitter to eth0
tc qdisc add dev eth0 root netem delay 200ms 50ms

# Test for 5 minutes then remove
sleep 300
tc qdisc del dev eth0 root
```

Resource Exhaustion:
- Fill disk space to test handling
- Consume memory to trigger OOM behavior
- CPU stress testing
- Database connection pool exhaustion

Chaos Testing Tools:
- Chaos Toolkit for orchestration
- Custom Ansible playbooks
- Automated via CI/CD on schedule

10. API Design

10.1 RESTful API Standards

Base URL:

```
https://api.nyotapay.com/v1
```

Versioning:
- URI versioning: `/v1/`, `/v2/`
- Header versioning: `Accept: application/vnd.nyotapay.v1+json`

Resource Naming:

```
GET    /customers              # List customers
POST   /customers              # Create customer
GET    /customers/{id}         # Get customer
PUT    /customers/{id}         # Update customer
DELETE /customers/{id}         # Delete customer

GET    /customers/{id}/accounts        # Nested resources
POST   /transfers/p2p                  # Action resources
POST   /transfers/{id}/cancel          # Sub-actions
```

10.2 Request/Response Format

Standard Request:

```
POST /api/v1/transfers/p2p
Content-Type: application/json
Authorization: Bearer eyJhbGc...
X-Idempotency-Key: 550e8400-e29b-41d4-a716-446655440000
X-Request-ID: 7c9e6679-7425-40de-944b-e07fc1f90ae7

{
  "sourceAccountId": "550e8400-e29b-41d4-a716-446655440000",
  "destinationAccountId": "6ba7b810-9dad-11d1-80b4-00c04fd430c8",
  "amount": 50000,
  "currency": "TZS",
  "reference": "PAYMENT-12345",
  "description": "Payment for services",
  "metadata": {
    "channel": "mobile_app",
    "deviceId": "abc123"
  }
}
```

Success Response (202 Accepted):

```
HTTP/1.1 202 Accepted
Content-Type: application/json
X-Request-ID: 7c9e6679-7425-40de-944b-e07fc1f90ae7
Location: /api/v1/transfers/7c9e6679-7425-40de-944b-e07fc1f90ae7

{
  "transferId": "7c9e6679-7425-40de-944b-e07fc1f90ae7",
  "status": "PROCESSING",
  "message": "Transfer is being processed",
  "statusUrl": "/api/v1/transfers/7c9e6679-7425-40de-944b-e07fc1f90ae7",
  "estimatedCompletion": "2025-11-05T10:35:00Z",
  "_links": {
    "self": { "href": "/api/v1/transfers/7c9e6679-7425-40de-944b-e07fc1f90ae7" },
    "cancel": { "href": "/api/v1/transfers/7c9e6679-7425-40de-944b-e07fc1f90ae7/cancel" }
  }
}
```

Error Response (400 Bad Request):

```
HTTP/1.1 400 Bad Request
Content-Type: application/problem+json
X-Request-ID: 7c9e6679-7425-40de-944b-e07fc1f90ae7

{
  "type": "https://api.nyotapay.com/errors/insufficient-balance",
  "title": "Insufficient Balance",
  "status": 400,
  "detail": "Account balance is insufficient for this transfer",
  "instance": "/api/v1/transfers/p2p",
  "traceId": "7c9e6679-7425-40de-944b-e07fc1f90ae7",
  "errors": [
    {
      "field": "amount",
      "message": "Amount exceeds available balance of 30000 TZS"
    }
  ],
  "availableBalance": 30000,
  "requestedAmount": 50000
}
```

10.3 Idempotency

Idempotency Key Handling:

Ensures duplicate requests return the same result without re-processing:

Process Flow:
1. Client includes unique idempotency key in request header (X-Idempotency-Key)
2. Service checks if key exists in idempotency store
3. **If key exists**: Return previously stored response (status code and body)
4. **If key is new**:
- Process the transfer request
- Store result with idempotency key
- Set 24-hour expiration on stored result
- Return accepted response

Key Features:
- Prevents duplicate transactions from network retries
- Returns identical response for duplicate requests
- 24-hour retention window for idempotency keys
- Stored response includes status code and full response body

10.4 Pagination

Cursor-Based Pagination:

```
GET /api/v1/transactions?limit=20&cursor=eyJpZCI6MTIzNDV9

Response:
{
  "data": [...],
  "pagination": {
    "nextCursor": "eyJpZCI6MTIzNjV9",
    "hasMore": true,
    "count": 20
  },
  "_links": {
    "next": { "href": "/api/v1/transactions?limit=20&cursor=eyJpZCI6MTIzNjV9" }
  }
}
```

10.5 Webhooks

Webhook Registration:

```
POST /api/v1/webhooks
{
  "url": "https://merchant.com/webhooks/nyotapay",
  "events": [
    "transfer.completed",
    "transfer.failed",
    "payment.completed"
  ],
  "secret": "whsec_abc123..."
}
```

Webhook Payload:

```
POST https://merchant.com/webhooks/nyotapay
Content-Type: application/json
X-NyotaPay-Signature: sha256=abc123...
X-NyotaPay-Event: transfer.completed

{
  "eventId": "evt_550e8400",
  "eventType": "transfer.completed",
  "timestamp": "2025-11-05T10:35:00Z",
  "data": {
    "transferId": "7c9e6679-7425-40de-944b-e07fc1f90ae7",
    "status": "COMPLETED",
    "amount": 50000,
    "currency": "TZS"
  }
}
```

Signature Verification:

Webhooks include HMAC-SHA256 signature for authentication:

Verification Process:
1. Extract signature from X-NyotaPay-Signature header
2. Retrieve webhook secret for the merchant
3. Compute HMAC-SHA256 hash of raw payload using secret
4. Format computed hash as "sha256={hex_string}"
5. Compare computed signature with received signature (case-insensitive)
6. Reject webhook if signatures don't match

Security Benefits:
- Verifies webhook came from NyotaPay
- Prevents tampering with webhook data
- Protects against replay attacks when combined with timestamp checking

11. Observability

11.1 Three Pillars

Metrics → Logs → Traces

11.2 Metrics (Prometheus + Grafana)

Application Metrics:

The system collects metrics using Prometheus client libraries:

Counter Metrics:
- Track total transfer requests
- Labels: type (p2p, cash-in, merchant), status (success, error)
- Increment on each request
- Example: `nyotapay_transfer_requests_total{type="p2p",status="success"}`

Histogram Metrics:
- Measure transfer processing duration
- Labels: transaction type
- Buckets: 0.1s, 0.5s, 1s, 2s, 5s, 10s
- Calculate percentiles (P50, P95, P99)
- Example: `nyotapay_transfer_duration_seconds`

Gauge Metrics:
- Track current number of active transfers
- Updates in real-time as transfers start/complete
- Example: `nyotapay_active_transfers`

Usage Pattern:
- Increment counters on events
- Record histogram observations with timing
- Set gauge values for current state

Infrastructure Metrics:
- CPU, Memory, Disk usage
- Network I/O
- Pod restarts
- Request rate, error rate, duration (RED)

Business Metrics:
- Transaction volume
- Revenue (by type, by merchant)
- Customer acquisition
- Churn rate

Grafana Dashboards:
- Service overview (SLIs, error rate, latency)
- Transaction monitoring
- Infrastructure health
- Business KPIs

11.3 Logging (ELK Stack / Loki)

Structured Logging:

All services use structured logging with key-value pairs for easy querying:

Log Entry Components:
- **Timestamp**: ISO 8601 format with milliseconds
- **Level**: Information, Warning, Error, etc.
- **Message**: Human-readable description
- **Context Fields**: Key business data (transferId, accountId, amount)
- **Request ID**: Correlation ID for tracing
- **Service Metadata**: Service name, version, environment

Example Log Output (JSON format):

```
{
  "@timestamp": "2025-11-05T10:30:00.123Z",
  "level": "Information",
  "message": "Transfer initiated",
  "transferId": "7c9e6679-7425-40de-944b-e07fc1f90ae7",
  "sourceAccount": "550e8400-e29b-41d4-a716-446655440000",
  "amount": 50000,
  "requestId": "7c9e6679-7425-40de-944b-e07fc1f90ae7",
  "service": "p2p-service",
  "version": "1.2.0",
  "environment": "production"
}
```

Benefits:
- Easy filtering and searching in log aggregation tools
- Consistent format across all services
- Rich context for debugging

Log Levels:
- **TRACE**: Very detailed (dev only)
- **DEBUG**: Diagnostic information (dev/staging)
- **INFO**: General operational events
- **WARN**: Recoverable issues
- **ERROR**: Unhandled errors
- **FATAL**: Critical failures

Centralized Logging:

```
Application → Fluent Bit → Elasticsearch → Kibana
              (sidecar)    (index)         (visualize)
```

11.4 Distributed Tracing (Jaeger/Zipkin)

Trace Context Propagation:

Distributed tracing tracks requests across all services:

Trace Implementation:
1. Start activity (span) for each operation with descriptive name
2. Add tags with business context:
- Transfer ID
- Transaction type
- Amount
- Account IDs
3. Execute business logic
4. Add tags for key milestones (e.g., balance reserved)
5. Set final status:
- **Success**: Mark as OK with completion time

- **Error**: Mark as Error, record exception details
6. Activity automatically propagates to downstream services

Trace Hierarchy:
- Parent span: HTTP request handling
- Child spans: Database queries, external API calls, business logic
- Each span includes timing and metadata

Error Handling:
- Exceptions are recorded with full stack trace
- Error status propagates up the call chain
- Helps identify exact failure point

Trace Visualization:

```
[API Gateway] → [P2P Service] → [Wallet Service] → [Ledger Service]
     |              |                  |                  |
   10ms           25ms               15ms               30ms
                                                          |
                                                    [SQL Server]
                                                        50ms
Total: 130ms
```

Key Traces:
- Request-to-response flow
- Cross-service calls
- Database queries
- External API calls
- Error paths

11.5 Alerting

Prometheus Alert Rules:

```
groups:
- name: nyotapay_alerts
  interval: 30s
  rules:
  - alert: HighErrorRate
    expr: |
      rate(nyotapay_transfer_requests_total{status="error"}[5m]) > 0.05
    for: 2m
    labels:
      severity: critical
    annotations:
      summary: "High error rate detected"
      description: "Error rate is {{ $value | humanizePercentage }} for {{ $labels.type }}"

  - alert: SlowTransferProcessing
    expr: |
      histogram_quantile(0.95,
        rate(nyotapay_transfer_duration_seconds_bucket[5m])
      ) > 5
    for: 5m
    labels:
      severity: warning
    annotations:
      summary: "Slow transfer processing"
      description: "P95 latency is {{ $value }}s"

  - alert: ServiceDown
    expr: up{job="p2p-service"} == 0
    for: 1m
    labels:
      severity: critical
    annotations:
      summary: "Service is down"
```

Alert Channels:
- PagerDuty (critical, on-call)
- Slack (warning, info)
- Email (daily digest)
- SMS (critical only)

11.6 SLIs & SLOs

Service Level Indicators:
- **Availability:** % of successful requests
- **Latency:** P95 response time < 500ms
- **Error Rate:** < 0.1% of requests fail
- **Throughput:** Requests per second

Service Level Objectives:

```
P2P Transfer Service:
- Availability: 99.95% (21.6 minutes downtime/month)
- Latency (P95): < 500ms
- Latency (P99): < 1000ms
- Error Rate: < 0.05%

Wallet Service:
- Availability: 99.99% (4.32 minutes downtime/month)
- Latency (P95): < 100ms
- Error Rate: < 0.01%
```

Error Budget:

```
Monthly Error Budget = (1 - SLO) × Total Requests
Example:
  SLO = 99.95%
  Total Requests = 10M
  Error Budget = 0.0005 × 10M = 5,000 failed requests
```

12. Disaster Recovery

12.1 Backup Strategy

Database Backups:
- **Full Backup:** Daily at 2 AM
- **Differential Backup:** Every 6 hours
- **Transaction Log Backup:** Every 15 minutes
- **Retention:** 7 days local, 30 days offsite, 7 years archived

Event Store Backups:
- **Snapshot:** Daily
- **Event Log:** Continuous replication
- **Retention:** Indefinite (legal requirement)

Configuration Backups:
- Infrastructure as Code (Git)
- Secrets (encrypted backups)
- Ansible playbooks and configurations (Git)
- VM templates and snapshots

12.2 Recovery Procedures

RTO (Recovery Time Objective):
- Critical services: 15 minutes
- Non-critical services: 1 hour
- Full system: 4 hours

RPO (Recovery Point Objective):
- Transactional data: 1 minute (transaction log backup)
- Configuration: Real-time (Git)

Disaster Scenarios:

1. Database Failure:

```
1. Failover to secondary replica (automatic, <30s)
2. Verify data consistency
3. Investigate and fix primary
4. Failback when stable
```

2. Availability Zone Failure:

```
1. HAProxy health checks detect failed VMs immediately
2. Traffic automatically rerouted to healthy VMs in other zones
3. VMs automatically restarted on healthy hosts (vSphere HA)
4. Provision replacement VMs if host hardware failed (5-10 min)
5. Monitor and scale if needed
```

3. Complete Data Center Failure:

```
1. Activate DR site
2. Restore latest backups
3. Replay transaction logs
4. Update DNS to DR site
5. Resume operations
Estimated: 2-4 hours
```

12.3 Business Continuity

Cold Standby:
- Infrastructure provisioned but not running
- Database backups replicated
- Manual activation process

Warm Standby:
- Minimal infrastructure running

- Database replication active
- Quick scale-up capability

Hot Standby (Recommended):
- Full infrastructure running
- Active-active database replication
- Automatic failover
- Geo-distributed deployment

12.4 Testing

DR Drills:
- Quarterly full DR test
- Monthly failover test
- Weekly backup restore test

Chaos Engineering:
- Pod failure injection
- Network latency/partition
- Database connection loss
- Resource exhaustion

Appendix

A. Technology Stack

Backend Services:
- .NET 8 / ASP.NET Core
- C# 12

Databases:
- SQL Server 2022 (primary)
- Redis 7 (cache)
- EventStoreDB (event sourcing)

Messaging:
- Apache Kafka / RabbitMQ

API Gateway:
- Kong / Apigee

Service Discovery & Load Balancing:
- Consul for service discovery

- HAProxy for load balancing
- Keepalived for HA

Virtualization & Orchestration:
- VMware vSphere / Hyper-V / Proxmox
- Terraform for VM provisioning
- Ansible for configuration management

Observability:
- Prometheus + Grafana
- ELK Stack / Loki
- Jaeger / Tempo

CI/CD:
- GitLab CI / GitHub Actions / Jenkins
- Ansible (Configuration Management)
- Terraform (Infrastructure Provisioning)

Security:
- HashiCorp Vault
- Cert-Manager
- OAuth 2.0 / OpenID Connect

B. Glossary

- **Aggregate:** DDD pattern representing a cluster of objects treated as a single unit
- **CQRS:** Command Query Responsibility Segregation
- **Event Sourcing:** Storing state changes as events
- **Idempotency:** Same operation produces same result regardless of repetition
- **Saga:** Pattern for managing distributed transactions
- **Service Mesh:** Infrastructure layer for service-to-service communication
- **SLI/SLO/SLA:** Service Level Indicator/Objective/Agreement

Document Control

Version	Date	Author	Changes
1.0	2024-10-01	Architecture Team	Initial version
2.0	2025-11-05	Architecture Team	Complete redesign with DDD, CQRS, Event Sourcing

END OF DOCUMENT